

SIMATS ENGINEERING

ASSESSMENT TOOL 1

Experiments

COURSE NAME: Computer Networks

COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student

Experiments

QUESTIONS:4

Total Marks:40

1. Capture packets during a TCP connection setup. Identify the three packets involved in the handshake (SYN, SYN-ACK, ACK). Demonstrate the role of each flag and how they establish a reliable connection. (BL4) (10)
2. Capture a DNS query using UDP in Wireshark. Compare the UDP header size with TCP and note the absence of sequence numbers or acknowledgments. Discuss how this lightweight design impacts speed and reliability. (BL4) (10)
3. Simulate packet loss (e.g., disconnect briefly) and capture traffic in Wireshark. Observe how TCP retransmits lost packets while UDP does not. Explain why this difference makes TCP reliable but slower compared to UDP. (BL4) (10)
4. Use Wireshark to capture DNS traffic while resolving a domain name. Identify the query type (A, AAAA, MX) and the response received. Measure the time taken for resolution and explain why DNS typically uses UDP.(BL4) (10)

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Question 1 — TCP Three-Way Handshake (10 Marks)

Capture packets during a TCP connection setup. Identify the three packets involved in the handshake (SYN, SYN-ACK, ACK). Demonstrate the role of each flag and how they establish a reliable connection.

1.1 Objective

To use Wireshark to capture live traffic during the establishment of a TCP connection, identify the SYN, SYN-ACK and ACK segments that make up the three-way handshake, and explain how the flags and sequence/acknowledgment numbers in each segment cooperate to establish a reliable, connection-oriented session between a client and a server.

1.2 Tools / Software Required

- Wireshark (packet capture and analysis)
- A host machine with an active network interface (Wi-Fi/Ethernet)
- A web browser or terminal utility (curl / telnet) to generate a fresh TCP connection, e.g. to a website on port 80/443

1.3 Procedure

1. Open Wireshark and select the active network interface (e.g. Wi-Fi or Ethernet).
2. Start a capture, and apply the display filter `tcp.flags.syn == 1 || (ip.addr == <server_ip> && tcp)` to isolate the connection of interest.
3. In a browser or terminal, initiate a new connection to a web server that is not already cached/connected, e.g. open `http://example.com` or run `curl http://example.com`.
4. Stop the capture and locate the first three packets exchanged between the client and the server for that connection.
5. Click each packet and expand the 'Transmission Control Protocol' section in the packet-details pane to inspect the Flags, Sequence Number and Acknowledgment Number fields.
6. Right-click the first SYN packet and choose Follow → TCP Stream to see the whole conversation isolated from other traffic.

1.4 Sample Captured Output

A representative capture of a handshake between a client (192.168.1.10) and a web server (93.184.216.34) is summarised below. Replace this with the values from your own capture.

No.	Time	Source	Destination	Flags	Seq / Ack	Info
1	0.000000	192.168.1.10	93.184.216.34	SYN	Seq=0	Client → Server: 49514 → 80 [SYN] Seq=0 Win=64240 MSS=1460

2	0.028417	93.184.216.34	192.168.1.10	SYN, ACK	Seq=0 Ack=1	Server → Client: 80 → 49514 [SYN, ACK] Seq=0 Ack=1 Win=65535 MSS=1460
3	0.028562	192.168.1.10	93.184.216.34	ACK	Seq=1 Ack=1	Client → Server: 49514 → 80 [ACK] Seq=1 Ack=1 Win=64240

1.5 Role of Each Flag / Field

Packet	Flag(s) Set	Purpose
Packet 1 (Client → Server)	SYN = 1	Client proposes to open a connection and announces its Initial Sequence Number (ISN); no data is carried.
Packet 2 (Server → Client)	SYN = 1, ACK = 1	Server acknowledges the client's SYN (Ack = client ISN + 1) and simultaneously sends its own SYN with its own ISN, proposing the reverse direction of the connection.
Packet 3 (Client → Server)	ACK = 1	Client acknowledges the server's SYN (Ack = server ISN + 1); the connection is now ESTABLISHED and application data can flow.

1.6 How the Handshake Establishes Reliability

- Sequence & acknowledgment numbers: each side announces a random Initial Sequence Number and the other side must echo back ISN+1, proving both endpoints agree on the starting point for byte-level ordering — this is what lets TCP detect loss, duplication and re-ordering later in the session.
- Mutual capability negotiation: options carried in the SYN/SYN-ACK (MSS, Window Scale, SACK-permitted) let each side advertise its buffer size and segment limits, so data transfer afterwards is tuned to both endpoints.
- Full-duplex confirmation: because the handshake requires both a SYN and an ACK from each side, it confirms that packets can actually travel in both directions before any data is risked — this avoids the 'half-open' problem UDP cannot detect at all.
- Connection-state synchronisation: after step 3, both operating systems move the socket into the ESTABLISHED state, allocate buffers, and start tracking retransmission timers — the foundation for TCP's reliable, in-order delivery guarantee.

1.7 Analysis / Discussion

The three-way handshake (SYN → SYN-ACK → ACK) is often summarised as 'I want to talk / I hear you and I want to talk too / I hear you' — an odd number of messages is required precisely because acknowledgment must flow in the direction opposite to each SYN. Skipping any one of

the three packets is dangerous: relying on only the first two messages (a 'two-way' handshake) would let a client that never intended to connect (e.g. a spoofed source address) cause the server to commit resources — this is exactly the vulnerability exploited by a SYN-flood attack, which is why real-world stacks use SYN-cookies as a defence. The measured round-trip time between packet 1 and packet 3 (≈ 0.028 s in the sample above) is also the minimum latency any TCP application will experience before it can send its first byte of data, which is an important factor when comparing TCP-based protocols (HTTP, FTP, SSH) against connectionless UDP-based ones later in this report.

1.8 Conclusion

The capture confirms that TCP uses a three-segment SYN / SYN-ACK / ACK exchange to synchronise sequence numbers, negotiate connection parameters and verify bidirectional reachability before any data is exchanged, which is the mechanism underlying TCP's reliable, connection-oriented service model.

Question 2 — DNS over UDP vs. TCP Header Comparison (10 Marks)

Capture a DNS query using UDP in Wireshark. Compare the UDP header size with TCP and note the absence of sequence numbers or acknowledgments. Discuss how this lightweight design impacts speed and reliability.

2.1 Objective

To capture a DNS lookup carried over UDP, examine the UDP header fields in Wireshark, contrast the header structure and size with a TCP header, and reason about the speed/reliability trade-off that results from UDP's minimal, stateless design.

2.2 Tools / Software Required

- Wireshark
- Command-line DNS tools: `nslookup` or `dig` (`dig purge/+notcp` to force UDP)

2.3 Procedure

7. Start a Wireshark capture and apply the filter `udp.port == 53` (or simply `dns`).
8. Flush the local DNS cache (`ipconfig /flushdns` on Windows, `sudo systemd-resolve --flush-caches` on Linux) so the query is not answered from cache.
9. Run `nslookup www.example.com` (or `dig www.example.com`) from a terminal.
10. In Wireshark, select the outgoing DNS query packet and expand 'User Datagram Protocol' in the packet-details pane to inspect the UDP header fields.
11. For comparison, select any TCP segment already captured (e.g. from Question 1) and expand 'Transmission Control Protocol' to view its header fields side-by-side.

2.4 Sample Captured Output

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.10	8.8.8.8	DNS	70	Standard query 0x1a2b A www.example.com
2	0.021340	8.8.8.8	192.168.1.10	DNS	86	Standard query response 0x1a2b A www.example.com A 93.184.216.34

2.5 UDP Header vs. TCP Header

Field	UDP Header	TCP Header
Fixed header size	8 bytes	20 bytes (minimum, no options)
Source / Destination Port	Present (2 + 2 bytes)	Present (2 + 2 bytes)
Sequence Number	Absent	Present (4 bytes) — orders every byte sent
Acknowledgment Number	Absent	Present (4 bytes) — confirms received bytes
Flags (SYN/ACK/FIN/RST...)	Absent	Present (control bits for connection state)
Window Size (flow control)	Absent	Present (2 bytes)
Checksum	Present (optional in IPv4, 2 bytes)	Present (mandatory, 2 bytes)
Connection setup required	No — connectionless	Yes — 3-way handshake before data

2.6 Observations

- The captured DNS query/response used a total UDP header overhead of only 8 bytes per packet, versus the 20+ byte TCP header that Question 1's handshake packets carried — roughly 2.5× less overhead per packet.
- Because UDP has no Sequence Number or Acknowledgment Number field, the receiver has no built-in way to detect a missing, duplicated or out-of-order datagram; the DNS resolver instead relies on the 16-bit Query/Transaction ID inside the DNS payload itself to match a response to its request, and on an application-level timeout/retry if no response arrives.
- No handshake packets appear before the query — the very first packet in the capture is the DNS question itself, unlike Question 1 where three packets were needed before any data could be sent.

2.7 Impact of the Lightweight Design on Speed and Reliability

Speed

Because UDP skips connection setup and per-packet acknowledgment, a DNS lookup normally completes in a single round trip (one query packet out, one response packet back) — as reflected in the ≈ 21 ms gap between packets 1 and 2 above. A TCP-based exchange for the same data would need at least the 3-way handshake round trip plus the data round trip, roughly doubling the minimum latency. This is precisely why DNS, which is latency-sensitive and typically exchanges very small messages, defaults to UDP.

Reliability

The trade-off is that UDP provides no guarantee of delivery, ordering, or duplicate protection at the transport layer. If a DNS query or response datagram is lost, corrupted, or arrives out of order, UDP itself does nothing about it — the resolver application must implement its own timeout and retransmit logic, and correctness of matching a response to a request depends entirely on the DNS Transaction ID rather than TCP sequence numbers. For most DNS answers, which fit in a single small datagram, this simplicity is an acceptable trade because the cost of an occasional retried query is much lower than the added latency TCP would introduce on every lookup. DNS does fall back to TCP for responses that exceed the UDP/EDNS0 payload size (e.g. DNSSEC-signed answers or zone transfers), since TCP's segmentation and reliable delivery are needed once a message can no longer fit in one datagram.

2.8 Conclusion

The 8-byte UDP header, compared with TCP's 20-byte minimum header, confirms that UDP's design deliberately omits sequencing, acknowledgment and flow-control fields to minimise overhead and latency. This makes UDP well suited to DNS, where small, latency-sensitive request/response exchanges are common, at the cost of pushing reliability responsibilities up into the application layer.

Question 3 — Packet Loss: TCP Retransmission vs. UDP (10 Marks)

Simulate packet loss (e.g., disconnect briefly) and capture traffic in Wireshark. Observe how TCP retransmits lost packets while UDP does not. Explain why this difference makes TCP reliable but slower compared to UDP.

3.1 Objective

To induce packet loss during an active TCP file/data transfer and during a UDP stream, capture both in Wireshark, and compare how each protocol behaves — specifically, to observe TCP's retransmission mechanism and confirm that UDP performs no such recovery.

3.2 Tools / Software Required

- Wireshark

- A sustained TCP transfer to observe (e.g. downloading a large file with curl/wget, or streaming a video)
- A UDP stream to observe (e.g. an online UDP-based speed test, a VoIP/video call, or iperf3 -u between two hosts)
- A way to induce brief loss: toggling Wi-Fi off/on for 1-2 seconds, temporarily unplugging Ethernet, or using a network-conditioning tool (e.g. Clumsy on Windows, tc netem on Linux) to drop a percentage of packets

3.3 Procedure

12. Start a Wireshark capture on the active interface.
13. Begin a large TCP download (e.g. `curl -O <large-file-url>`) or start a video call/stream.
14. While the transfer is running, briefly disable and re-enable the network adapter (or apply artificial packet loss with a conditioning tool) for 1–2 seconds, then let the transfer continue.
15. Repeat the same brief-disconnect test while a UDP stream (e.g. iperf3 -u or a live video call) is running, in a separate capture.
16. Stop each capture and apply the filter `tcp.analysis.retransmission` on the TCP capture to isolate retransmitted segments, and inspect the UDP capture for any recovery packets (there will be none).
17. Use Statistics → I/O Graph in Wireshark to visualise the drop in throughput and the recovery period for both captures.

3.4 Sample Captured Output — TCP (retransmission visible)

No.	Time	Source → Dest	Info
145	5.201	192.168.1.10 → 93.184.216.34	[TCP Retransmission] Seq=48001 Len=1460
146	5.203	192.168.1.10 → 93.184.216.34	[TCP Retransmission] Seq=49461 Len=1460
150	5.640	93.184.216.34 → 192.168.1.10	[TCP Dup ACK] Ack=48001
162	6.812	93.184.216.34 → 192.168.1.10	ACK Ack=52381 (transfer resumes normally)

3.5 Sample Captured Output — UDP (no recovery)

No.	Time	Source → Dest	Info
88	5.150	192.168.1.10 → 192.168.1.20	UDP Src Port: 5004 Dst Port: 5004 Len=172

—	5.170–6.400	(gap in capture — no packets)	Packets sent during the outage are simply never seen; no retransmission entries exist
89	6.410	192.168.1.10 → 192.168.1.20	UDP Src Port: 5004 Dst Port: 5004 Len=172 (stream continues; lost data is skipped, not resent)

3.6 Observations

- In the TCP capture, once the link returned, the sender's unacknowledged segments (identified because no ACK arrived within the retransmission timeout, RTO) were automatically re-sent — Wireshark's expert system flags these as [TCP Retransmission] and the receiver's repeated [TCP Dup ACK] packets show it was still waiting for the missing byte range.
- Throughput visibly dips to zero during the outage and only recovers after the missing segments are retransmitted and acknowledged — the I/O graph shows a stall-and-recover ('sawtooth') pattern typical of TCP congestion control reacting to loss.
- In the UDP capture, the datagrams sent during the outage simply do not appear anywhere in the trace — there is no retransmission, no gap-filling, and no indication in the protocol itself that anything was lost. The stream resumes with new data as soon as the link returns; whatever was lost is gone permanently.

3.7 Why This Makes TCP Reliable but Slower

TCP's reliability comes directly from the state it keeps: every byte is numbered, the receiver acknowledges what it has received, and the sender starts a retransmission timer for anything unacknowledged. When the timer expires (or three duplicate ACKs arrive), TCP assumes loss and re-sends the segment, guaranteeing that the receiving application eventually gets every byte, in order, exactly once. This guarantee is not free — retransmission introduces waiting time (at least one RTT, often more if multiple rounds of loss occur), and TCP's congestion control additionally shrinks its sending rate after loss (e.g. halving the congestion window) as a precaution against network overload, further reducing throughput for a period after the event.

UDP has none of this machinery: it has no sequence numbers to detect a gap, no acknowledgments to know something arrived, and no timer to trigger a resend. This is a design choice, not an omission — real-time applications such as VoIP, video calls and live streaming would rather skip a lost 20 ms audio frame and keep the conversation flowing in real time than pause the entire stream to wait for a retransmission that would arrive too late to be useful. This is exactly the trade-off DNS also exploits in Question 2, and it explains why latency-sensitive, loss-tolerant traffic favours UDP while correctness-sensitive traffic (web pages, file transfers, email) favours TCP.

3.8 Conclusion

The controlled outage confirms that TCP detects and recovers from packet loss through retransmission and duplicate-ACK signalling, guaranteeing complete and ordered delivery at the

cost of added delay, whereas UDP passes losses straight through to the application with no recovery, trading reliability for lower latency and simpler processing.

Question 4 — DNS Query Types, Response, and Resolution Time (10 Marks)

Use Wireshark to capture DNS traffic while resolving a domain name. Identify the query type (A, AAAA, MX) and the response received. Measure the time taken for resolution and explain why DNS typically uses UDP.

4.1 Objective

To capture the DNS traffic generated while resolving a domain name, identify the resource-record query type(s) used and the answers returned, measure the query-to-response resolution time from the Wireshark timestamps, and explain the rationale for DNS's default use of UDP.

4.2 Tools / Software Required

- Wireshark
- dig (supports explicit record-type queries, e.g. `dig example.com A` , `dig example.com AAAA` , `dig example.com MX`)

4.3 Procedure

18. Start a Wireshark capture with the filter `dns` applied.
19. Flush the local DNS cache to force a fresh lookup.
20. Run three separate lookups to generate different query types: `dig example.com A` , `dig example.com AAAA` , and `dig example.com MX` .
21. Stop the capture. For each query, select the request packet and note its 'Time' column value, then select the matching response packet (same Transaction ID) and note its 'Time' value.
22. Expand each DNS layer in the packet-details pane to read the Queries and Answers sections, confirming the Type field (A / AAAA / MX) and the returned record.
23. Compute the resolution time for each query as (response timestamp – query timestamp).

4.4 Sample Captured Output

No.	Time (s)	Source → Dest	Query Type	Info
1	0.000000	192.168.1.10 → 8.8.8.8	A	Standard query 0x5c11 A example.com
2	0.024118	8.8.8.8 → 192.168.1.10	A	Standard query response 0x5c11 A example.com A 93.184.216.34

3	1.201355	192.168.1.10 → 8.8.8.8	AAAA	Standard query 0x7d2a AAAA example.com
4	1.229840	8.8.8.8 → 192.168.1.10	AAAA	Standard query response 0x7d2a AAAA example.com AAAA 2606:2800:220:1:248:1893:25c8:1946
5	2.503010	192.168.1.10 → 8.8.8.8	MX	Standard query 0x9e44 MX example.com
6	2.541677	8.8.8.8 → 192.168.1.10	MX	Standard query response 0x9e44 MX example.com 0 . (no mail exchanger configured)

4.5 Resolution Time Measurement

Query Type	Query Sent (s)	Response Received (s)	Resolution Time
A (IPv4 address)	0.000000	0.024118	24.1 ms
AAAA (IPv6 address)	1.201355	1.229840	28.5 ms
MX (mail exchanger)	2.503010	2.541677	38.7 ms

4.6 Observations

- All three query types (A, AAAA, MX) were resolved in a single UDP request/response pair — no query needed to fall back to TCP, since every response fit within the standard UDP/EDNS0 payload size.
- The A query returned an IPv4 address in the Answers section, the AAAA query returned an IPv6 address, and the MX query returned the mail-exchanger record (or an empty/priority-0 answer for a domain with no configured mail server) — confirming that the same DNS protocol and packet format is reused for different resource-record types simply by changing the QTYPE field.
- Measured resolution times were all under 40 ms in this sample capture, consistent with a single UDP round trip to a nearby recursive resolver, and noticeably faster than the multi-packet TCP handshake overhead observed in Question 1.

4.7 Why DNS Typically Uses UDP

- Low latency for small messages: a DNS question and answer usually fit in a single small datagram, so a single UDP round trip is enough — avoiding the extra round trip a TCP handshake would add before any data could be exchanged, exactly as demonstrated in Question 2.

- High query volume, low per-query cost: DNS resolvers handle very large numbers of short-lived lookups; UDP avoids the memory and CPU cost of maintaining per-connection TCP state for each one.
- Simplicity of request/response matching: DNS already carries its own 16-bit Transaction ID inside the payload, so it does not need TCP's sequence/acknowledgment machinery to match a response to its request.
- Application-level retry is sufficient: if a UDP query is lost, the resolver simply re-sends the question after a short timeout — cheap enough that avoiding it on every lookup (by using TCP by default) would not be worth the added latency.
- TCP fallback when needed: DNS still uses TCP when a response would exceed the UDP payload limit (large DNSSEC-signed responses) or for zone transfers between name servers, since TCP's reliable, segmented delivery is required once a message can no longer fit in a single datagram.

4.8 Conclusion

The capture shows that different DNS record types (A, AAAA, MX) share the same lightweight query/response format, each typically resolved in a single UDP round trip within tens of milliseconds. This confirms that DNS's default choice of UDP directly serves its need for fast, high-volume, small-message lookups, with TCP reserved only for the exceptional cases where message size or transactional reliability requires it.

Overall Summary

Protocol Behaviour	TCP	UDP
Connection setup	3-way handshake (SYN/SYN-ACK/ACK) before data	None — first packet can carry data
Header overhead	20 bytes minimum	8 bytes
Reliability	Guaranteed, in-order delivery via seq/ack + retransmission	Best-effort; no delivery guarantee
Behaviour under loss	Detects and retransmits lost segments	Lost data is simply gone; app decides what to do
Typical use in this report	General reliable transfers (web, file transfer)	DNS lookups (A/AAAA/MX), latency-sensitive traffic
Speed vs. reliability trade-off	Slower, more reliable	Faster, less reliable